C# Example of Meta-Schema v1

Validate payloads with Amazon Product Type Definition meta-schema instances.

Example Validator Implementation for .NET

For C# .NET applications, the Newtonsoft Json.NET Schema library supports JSON Schema Draft 2019-09 and custom vocabularies. The following example demonstrates how to utilize the Newtonsoft Json.NET Schema library to validate payloads with instances of the *Amazon Product Type Definition Meta-Schema*. There is no requirement to use this specific library or the example implementation. Amazon does not provide technical support for third-party JSON Schema libraries and this is provided as an example only.

Schema Configuration

When using the Newtonsoft Json.NET Schema to validate instances of the *Amazon Product Type Definition Meta Schema* with custom vocabulary, the meta-schema is configured with a `JSchemaResolver` and custom keyword validation is configured with `JSchemaReaderSettings` when parsing instances of the *Amazon Product Type Definition Meta-Schema*.

Constants:

C#

```
// $id of the Amazon Product Type Definition Meta-Schema.
var schemaId = "https://schemas.amazon.com/selling-partners/definitions/product-types/meta-schema/v1";

// Local copy of the Amazon Product Type Definition Meta-Schema.
var metaSchemaPath = "./amazon-product-type-definition-meta-schema-v1.json";

// Local copy of an instance of the Amazon Product Type Definition Meta-Schema.
var luggageSchemaPath = "./luggage.json";
```

Configure Meta-Schema:

C#

```
// Schema resolver that uses local copies of schemas
rather than retrieving them from the web.
var resolver = new JSchemaPreloadedResolver();

// Add the meta-schema to the resolver.
resolver.Add(new Uri(schemaId),
File.ReadAllText(metaSchemaPath));

// Configure reader settings with resolver and
keyword validators to use when parsing instances of
the meta-schema.
var readerSettings = new JSchemaReaderSettings
    {
        Resolver = resolver,
        Validators = new List<JsonValidator>
        {
            new MaxUniqueItemsKeywordValidator(),
            new MaxUtf8ByteLengthKeywordValidator(),
            new MinUtf8ByteLengthKeywordValidator()
        }
    };
```

Load Meta-Schema Instance:

C#

```
var luggageSchema =
JSchema.Parse(File.ReadAllText(luggageSchemaPath),
readerSettings);
```

Payload Validation

With an instance of the *Amazon Product Type Definition Meta-Schema* loaded as a `JSchema` instance, payloads can be validated using the instance.

C#

```
// Create a JObject for the payload (this can be
constructed in code or read from a file).
var payload =
JObject.Parse(File.ReadAllText("./payload.json"));

// Validate the payload and get any resulting
validation messages.
var valid = payload.IsValid(luggageSchema, out
IList<string> errorMessages);
```

If the payload is valid, `IsValid` will return `true` with an empty list of error messages. Otherwise `IsValid` will return `false` with a list of error messages.

Keyword Validation

The Newtonsoft Json.NET Schema supports validating custom vocabulary by using classes that implementing the

Special Authorization Types

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

`JsonValidator` interface and provide the validation logic.

Refer to https://www.newtonsoft.com/jsonschema/help/html/CustomJsonValidators.htm.

The following examples illustrate implementations of the `JsonValidator` interface that validate the custom vocabulary in instances of the *Amazon Product Type Definition Meta-Schema*.

`MaxUniqueItemsKeyword` Class

C#

```
using System.Collections.Generic;
using System.Linq;
using Newtonsoft.Json.Linq;
using Newtonsoft.Json.Schema;

namespace AmazonProductTypeSchemaValidator
{
    /**
     * Example validator for the "maxUniqueItems"
keyword.
     */
    public class MaxUniqueItemsKeywordValidator :
JsonValidator
    {
        public override void Validate(JToken value,
JsonValidatorContext context)
        {
            var maxUniqueItems =
GetMaxUniqueItems(context.Schema);

            // Get the selector properties
configured on the scheme element, if they exist.
Otherwise, this validator
            // defaults to using all properties.
            var selectors =
GetSelectors(context.Schema);

            // Only process if the value is an array
with values.
            if (value.Type != JTokenType.Array)
return;

            // Create a property-value dictionary of
each items properties (selectors) and count the
number of
            // occurrences for each combination.
            var uniqueItemCounts = new
Dictionary<IDictionary<string, string>, int>(new
UniqueKeyComparer());
            foreach (var instance in value)
            {
                // Only process instances in the
array that are objects.
                if (instance.Type !=
JTokenType.Object) continue;

                var instanceObject =
JObject.FromObject(instance);
```

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors

Listings Items API Issues Troubleshooting

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

```
                var uniqueKeys =
instanceObject.Properties()
                    .Where(property =>
selectors.Count == 0 ||
selectors.Contains(property.Name))
                    .ToDictionary(property =>
property.Name, property =>
property.Value.ToString());
```

`MaxUtf8ByteLengthKeyword` Class

```
uniqueItemCounts.GetValueOrDefault(uniqueKeys, 0) +
1;
```

C#

```
using System.Text;
using Newtonsoft.Json.Linq;
using Newtonsoft.Json.Schema;

namespace AmazonProductTypeSchemaValidator
{
    /**
     * Example validator for the "maxUtf8ByteLength"
keyword.
     */
    public class MaxUtf8ByteLengthKeywordValidator :
JsonValidator
    {
        public override void Validate(JToken value,
JsonValidatorContext context)
        {
            var maxUtf8ByteLength =
GetMaxUtf8ByteLength(context.Schema);
            if
(Encoding.UTF8.GetBytes(value.ToString()).Length >
maxUtf8ByteLength)
                context.RaiseError($"Value must be
less than or equal {maxUtf8ByteLength} bytes in
length.");
        }

        public override bool CanValidate(JSchema
schema)
        {
            return GetMaxUtf8ByteLength(schema) >=
0;
        }

        private static int
GetMaxUtf8ByteLength(JSchema schema)
        {
            var schemaObject =
JObject.FromObject(schema);
            var byteLengthProperty =
schemaObject["maxUtf8ByteLength"];

            if (byteLengthProperty != null &&
int.TryParse(byteLengthProperty.ToString(), out var
maxUtf8ByteLength))
                return maxUtf8ByteLength;

            return -1;
        }
    }
}
```

Edit Your Appstore Listing
Check Listing Status
Appstore Ratings and Reviews
Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview
Amazon Selling Partner API Guard Implementation Guide
VAT Calculation Service
Amazon Seller Data Access
Best Practices

`MinUtf8ByteLengthKeyword` Class

```
            var maxUniqueItemsProperty =
C# schemaObject["maxUniqueItems"];
```

```
using System.Text;
using Newtonsoft.Json.Linq;
using Newtonsoft.Json.Schema;

namespace AmazonProductTypeSchemaValidator
{
    /**
     * Example validator for the "minUtf8ByteLength"
keyword.
     */
    public class MinUtf8ByteLengthKeywordValidator :
JsonValidator
    {
        public override void Validate(JToken value,
JsonValidatorContext context)
        {
            var minUtf8ByteLength =
GetMinUtf8ByteLength(context.Schema);
            if
(Encoding.UTF8.GetBytes(value.ToString()).Length <
minUtf8ByteLength)
                context.RaiseError($"Value must be
greater than or equal {minUtf8ByteLength} bytes in
length.");
        }

        public override bool CanValidate(JSchema
schema)
        {
            return GetMinUtf8ByteLength(schema) >=
0;
        }

        private static int
GetMinUtf8ByteLength(JSchema schema)
        {
            var schemaObject =
JObject.FromObject(schema);
            var byteLengthProperty =
schemaObject["minUtf8ByteLength"];

            if (byteLengthProperty != null &&
int.TryParse(byteLengthProperty.ToString(), out var
minUtf8ByteLength))
                return minUtf8ByteLength;

            return -1;
        }
    }
}
```

Updated 19 days ago

← Amazon Product Type Definitions Meta-Schema (v1)

Javascript Example of Meta-Schema v1 →

A+ CONTENT API

A+ Content API

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API

Amazon Warehousing and Distribution API v2024-05-09 Reference

APP INTEGRATIONS API

App Integrations API

APPLICATION MANAGEMENT API

Application Management API

CATALOG ITEMS API

Catalog Items API

Catalog Items API Rate Limits

Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API

Customer Feedback API Rate Limits

DATA KIOSK

Data Kiosk API >

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API >

EASY SHIP API

Easy Ship API >

EXTERNAL FULFILLMENT

External Fulfillment Inventory API >

External Fulfillment Returns API >

External Fulfillment Shipping API >

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

FBA Inventory Dynamic Sandbox Guide

FEEDS API

NOTIFICATIONS API

Notifications API

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification

ORDERS API

Orders API

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API

PRODUCT PRICING API

Product Pricing API

Product Pricing API and Notifications FAQ

Seller Wallet API Rate Limits

SELLERS API

Sellers API

SERVICES API

Services API

SHIPMENT INVOICING API

Shipment Invoicing API

SHIPPING API

Shipping API v2 Resources

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API

SUPPLY SOURCES API

Supply Sources API

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >